

L03b · CORALNPU 教学仓

L03b · M 扩展、CSR 与异常

作业说明 —— 任务拆解、验收标准与调试方法

对应上游仓库: [google/coralnpu](https://github.com/google/coralnpu) (RTL、文档、测试程序都来自这里)

学习方式: 读规格 → 写 RTL → `./learn check` 对拍 → 修复

文档版本: 2026-09-15 · 生成命令: `./learn pdf`

目录

L03b 作业说明

0. 交付物
1. 接口契约
2. 分步完成建议
3. 验收标准
4. 调试方法
5. 常见坑
6. 进阶挑战（不计入验收）
7. 学习记录（建议提交）

L03b 作业说明

0. 交付物

改两个文件（核心由课程提供，不要改）：

文件	TODO	内容
rtl/mdu.sv	7 个	8 条 M 扩展指令
rtl/csr_file.sv	2 个	CSR 读端口 + 写端口与异常提交

课程提供：tests/core_l03b.sv（流水线核 + MDU/CSR 接入 + 异常与 mret 的重定向）、tb_cpu.sv（L02 的测试平台，支持 +LATENCY）、三个测试程序。

复用的旧成果：L02 的 lsu.sv、L00 的 alu32.sv、L03a 的流水线（在课程提供的核里）。

验收：

```
./learn check L03b           # 18/18 (9 个程序 × 2 种延迟)
./learn check L03b --only 07  # 只跑 M 扩展
./learn check L03b --only 09  # 只跑异常
./learn check L03b --ref      # 参考实现
```

1. 接口契约

mdu.sv

MDU = Multiply-Divide Unit（乘除法单元），负责 M 扩展的 4 条乘法 + 4 条除法指令（上游 CoralNPU 把它们拆成 Mlu.scala 与 Dvu.scala，本课合并成一个模块）。

```
module mdu (
    input  logic [31:0] a, b,
    input  logic [2:0]  op,    // 就是指令的 funct3
    output logic [31:0] y
);
```

组合电路，一拍出结果。

csr_file.sv

```
module csr_file (
    input logic      clk, rst,
    input logic [11:0] raddr,      // 读地址 (EX 级组合读)
    output logic [31:0] rdata,
    input logic      we,          // WB 级提交时写
    input logic [11:0] waddr,
    input logic [31:0] wdata,
    input logic      trap_en,     // 异常提交
    input logic [31:0] trap_cause, trap_epc,
    output logic [31:0] mtvec_o, mepc_o
);
```

约定：

1. 读端口组合输出：`misa=0x40001100`、`mhartid=0`，未实现地址读 0；
2. 写端口在时钟沿生效；`misa/mhartid` 只读；
3. `trap_en` 有效时写 `mepc=trap_epc`、`mcause=trap_cause`、`mtval=0`，且优先级高于普通写。

2. 分步完成建议

Step 1: M 扩展的乘法（让 07 的前三条通过）

```
logic [63:0]      uprod;
logic signed [63:0] sprod, sprod_su;
assign uprod      = a * b;
assign sprod      = sa * sb;
assign sprod_su   = sa * $signed({1'b0, b});
```

然后按 op 取低 32 位或高 32 位。

Step 2: 除法与三个边界

```
3'b100: begin                                // div
    if (b == 0)                               y = 32'hFFFF_FFFF;
    else if (a == 32'h8000_0000 && b == 32'hFFFF_FFFF) y = 32'h8000_0000;
    else                                       y = sa / sb;
end
```

`divu/rem/remu` 同理，注意除零返回被除数的是 `rem/remu`。

自检：`./learn check L03b --only 07`。

Step 3: CSR 读端口

`rdata` 是组合输出，按 `raddr` 选择；`misa/mhartid` 返回常量。

自检：`./learn check L03b --only 08`（此时读出值为 0，会失败在“旧值”那一列）。

Step 4: CSR 写端口 + 异常提交

```
if (we) begin
    unique case (waddr)
        CSR_MSTATUS: mstatus <= wdata;
        ...
        default: ; // 只读寄存器忽略写入
    endcase
end
if (trap_en) begin // 优先级更高: 写在后面
    mepc <= trap_epc;
    mcause <= trap_cause;
    mtval <= 32'h0;
end
```

自检: `./learn check L03b` (18/18) 。

3. 验收标准

[通过] 07_mul_div.S lat0 / lat2

[通过] 08_csr.S lat0 / lat2

[通过] 09_exception.S lat0 / lat2

[通过] 01_arith.S ... 06_unaligned.S (回归)

18/18 次运行通过

4. 调试方法

检查器说	通常是
07 在 mulh 处失败, 期望 0xffffffff 实际 0x0	高位乘法没做, 或符号扩展写错
07 在 div 处失败, 实际是 x 或全 0	除零没挡住 (直接算了 a / b)
08 的"旧值" 一列全 0	CSR 读端口没写
08 的"新值" 一列不对	写端口没写, 或 set/clear 的位运算写反
08 中 csrrs ..., x0 之后值变了	忘了"rs1=x0 不写" 这条规定
09 超时、且反复执行同一条 pc	mepc 没被处理程序改 (死循环触发同一个异常)
09 跳到 mtvec 但 mcause 是 0	trap_en 没接、或 mcause 写错地址

辅助手段: 核心里有调试代码 (编译时加 `-DL03B_DEBUG`) , 会打印每次 CSR 访问的地址、读出值、待提交和目标地址, 非常适合定位异常流程:

```
iverilog -g2012 -DL03B_DEBUG -o /tmp/sim.vvp -s tb_cpu \
course/lessons/L02_lsu/tests/tb_cpu.sv \
course/lessons/L03b_mdu_csr/tests/core_l03b.sv \
course/lessons/L03b_mdu_csr/rtl/mdu.sv \
course/lessons/L03b_mdu_csr/rtl/csr_file.sv \
course/lessons/L02_lsu/rtl/lsu.sv course/lessons/L00_setup/rtl/alu32.sv
```

5. 常见坑

现象	根因	修法
<code>y = a / b</code> 结果出现 <code>x</code>	除零路径没挡住	先用 if 判断，再算除法
<code>mulh</code> 与 <code>mulhu</code> 结果一样	都用无符号（或都用有符号）算的	按 op 选不同的 64 位乘积
减法式取余符号错	用 <code>a % b</code> 而不是 <code>\$signed(a) % \$signed(b)</code>	操作数要是 signed
<code>csrw mepc</code> 紧跟 <code>mret</code> 不生效	CSR 数据冒险（EX 读到旧值）	核心已做 CSR 旁路；若你改动核心要保留
<code>csr rs rd, csr, x0</code> 改了 CSR	漏了“x0 不写”规定	<code>we = !(set/clear && rs1==0)</code>
异常返回后重新执行同一条指令	处理程序没改 mepc	<code>mepc = mepc + 4</code> （或按需修正）再 <code>mret</code>
用函数做 CSR 旁路时 iverilog 行为异常	iverilog 对“函数读模块信号”支持不好	用 <code>always_comb</code> 显式写（本课核心就是这么做的）

6. 进阶挑战（不计入验收）

- **挑战 1（多周期除法器）**：把组合除法改成 32 拍移位减的流水 divider，并像 LSU 那样在 EX/MEM 级插入“busy/done”握手（提示：`doc/microarch/mlu.md` 的 3 级流水乘法器接口）；
- **挑战 2（更多异常）**：加上访存地址越界（`mcause=5/7`）与 `mtval` 记录出错地址；
- **挑战 3（计数器）**：实现 `mcycle / minstret`（0xB00/0xB02）并让启动代码能读；
- **挑战 4（软件配合）**：写一个“除零检测”的处理程序：捕获异常、打印 `mcause`、让程序继续。

7. 学习记录（建议提交）

在 `course/work/L03b/记录.md` 里写下：

1. 你在 M 扩展的三个边界里，哪个一开始没写对；
2. 描述一次异常从 `ecall` 到 `mret` 的完整过程（用你自己的话，含寄存器变化）；
3. 对照上游的 `Mlu.scala / Csr.scala`，说出你的实现比它少什么。